RAW JAVA

# Student Management System

*A Comprehensive Java Application for Academic Record Management*

Core Java • OOP Concepts • File Handling • Collections

# Table of Contents


**01** **Project Overview**
*Introduction & Objectives*

**02** **System Features**
*Key Functionalities*

**03** **System Architecture**
*OOP Design & Modules*

**04** **Class Diagram**
*Java Classes & Relationships*

**05** **Core Code Snippets**
*Highlighted Java Code*

**06** **Technology Stack**
*Tools & Libraries Used*

**07** **Future Enhancements**
*Roadmap & Improvements*

**08** **Conclusion**
*Summary & Takeaways*

# 01 | Project Overview

## What is SMS?

A Student Management System is a Java console application that automates and manages student academic records digitally.

Core capabilities include:

- Add, update & delete student records
- Search & filter by various criteria
- Track marks and attendance data
- Persist data using File I/O streams
- Menu-driven interactive interface

## Project Objectives

### Apply OOP Principles

*Encapsulation, Inheritance, Polymorphism*

### Practice Java Collections

*ArrayList, HashMap for data storage*

### File Handling

*Read/write using FileReader, BufferedWriter*

### Modular Design

*Clean separation of classes & methods*

### User-Friendly Interface

*Console-based menu navigation system*

# 02 | System Features

+ S

### Add Student

Register new student with ID, name, age, course, marks and contact info

/ E

### Edit Record

Modify existing student details dynamically by Student ID

X D

### Delete Student

Remove student records safely with confirmation checks

Q S

### Search Student

Search by name, ID, or course using linear search algorithm

= L

### View All Records

Display all students in formatted tabular output in console

IO F

### File Persistence

Save & load data automatically using Java File I/O streams

# 03 | System Architecture

**PRESENTATION LAYER**

*MainMenu.java | ConsoleUI.java*

**BUSINESS LOGIC LAYER**

*StudentService.java | Validator.java*

**DATA ACCESS LAYER**

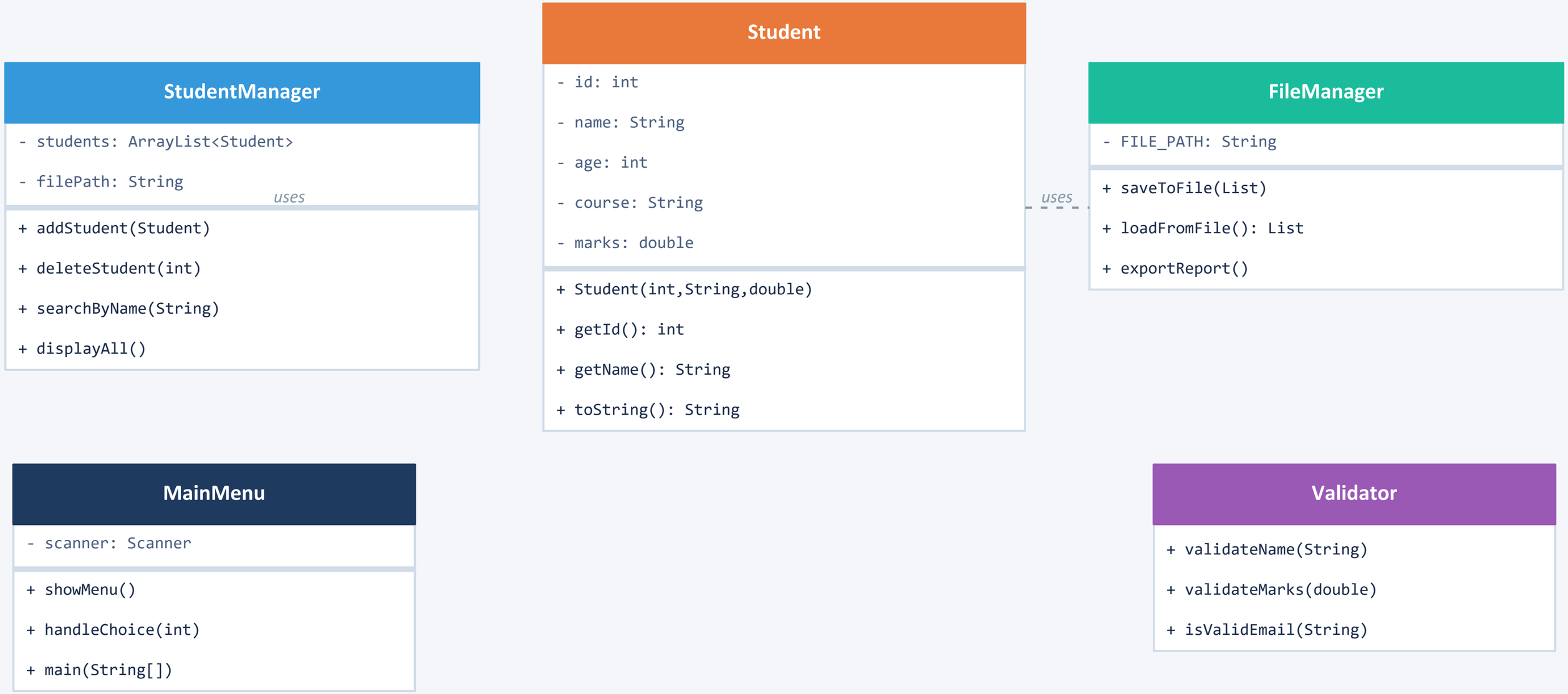
*StudentDAO.java | FileManager.java*

**DATA STORAGE**

*students.txt | File System*

*Clean 3-Tier Architecture ensures separation of concerns, maintainability, and future scalability.*

# 04 | Class Diagram



# 05 | Core Code Snippets

**Student.java**

```java
public class Student {

  private int id;

  private String name;

  private double marks;


  // Constructor

  public Student(int id, String name,
              double
marks) {

    this.id = id;

    this.name = name;

    this.marks = marks;

  }

  // Getters & Setters...

}
```

**StudentManager.java**

```java
public void addStudent(Student s) {

  students.add(s);

  fileManager.save(students);

  System.out.println(

    "Added: " + s.getName());

}


public Student searchById(int id) {

  for (Student s : students) {

    if (s.getId() == id)

      return s;

  }

  return null;
}
```

# 06 | Technology Stack

LANG

## Java SE 17+

Language

Core language with modern features

FRMW

## Java Collections

Framework

ArrayList, HashMap, Iterator, List

I/O

## Java File I/O

Library

FileReader, BufferedWriter, Scanner

IDE

## IntelliJ IDEA

IDE

JetBrains intelligent Java IDE

SDK

## JDK (Dev Kit)

Toolset

Compile, run and debug Java code

VCS

## Git & GitHub

Source Control

Version control & collaboration

# 07 | Future Enhancements

## Phase 1 — Near Term

1. GUI with Java Swing / JavaFX
2. MySQL database via JDBC
3. Login & role-based access

## Phase 2 — Mid Term

1. Attendance tracking module
2. GPA and grade calculator
3. PDF report generation (iText)

## Phase 3 — Long Term

1. Web-based via Spring Boot
2. REST API for mobile apps
3. Cloud deployment (AWS/GCP)

*Each phase builds toward a full-featured, enterprise-grade Student Information System.*

# Key Takeaways

- Demonstrates strong OOP principles in Java
- Modular, readable and maintainable code
- Real-world File I/O and data persistence
- Collections framework applied effectively
- Scalable architecture ready for upgrades

## Thank You!

**Student Management System**

*Built with Raw Java*

**Questions & Discussions**

**Open for Discussion**

*Java Project Presentation*